Temporal Graph Neural Networks for Real-Time Financial Fraud Detection in High-Velocity Transaction Streams

Alexandra M. Chen
Department of Computer Science and Financial Technology
Stanford University, Stanford, CA 94305, USA

Benjamin R. Thompson
Fraud Analytics Research Division
JP Morgan Chase AI Research, New York, NY 10017, USA

Sophia K. Watanabe
MIT Media Lab, Massachusetts Institute of Technology
Cambridge, MA 02139, USA

Abstract

The exponential growth of digital financial transactions has necessitated the development of sophisticated real-time fraud detection systems capable of identifying complex fraudulent patterns in high-velocity transaction streams. This research presents a comprehensive framework leveraging Temporal Graph Neural Networks (TGNs) to address the dual challenges of dynamic relational structure and temporal evolution inherent in financial transaction networks. We propose a novel architecture that integrates memory modules, temporal attention mechanisms, and adaptive message passing to process continuous-time dynamic graphs constructed from heterogeneous financial entities. Our system operates within strict latency constraints (<100ms) while maintaining superior detection accuracy compared to static graph and traditional machine learning approaches. Experimental evaluation on real-world financial datasets demonstrates that our TGN-based approach achieves an AUC-ROC of 0.974 and reduces false positives by 34.6% compared to state-of-the-art baselines. The framework effectively addresses critical challenges including concept drift, class imbalance, and adversarial camouflage behaviors through innovative techniques such as temporal neighborhood sampling and cost-sensitive learning. This research contributes both a theoretical advancement in temporal graph learning and a practical deployment blueprint for financial institutions seeking to implement next-generation fraud detection systems.

Keywords: Temporal Graph Neural Networks, Real-Time Fraud Detection, Dynamic Graph Learning, Financial Transaction Streams, Anomaly Detection

1. Introduction

The digitization of financial services has precipitated an unprecedented increase in transaction volumes, with global digital payment transactions projected to exceed $15 trillion by 2027. This exponential growth has been accompanied by sophisticated fraudulent activities that cause estimated annual losses of $40 billion worldwide. Traditional fraud detection systems, predominantly based on rule-based engines and static machine learning models, are increasingly inadequate against evolving fraud strategies that exploit temporal patterns and relational structures within financial networks . The limitations of these conventional approaches are particularly pronounced in high-velocity transaction environments where detection must occur within milliseconds to prevent financial loss while minimizing false positives that degrade customer experience .

The fundamental challenge in modern financial fraud detection lies in capturing the intricate interplay between temporal dynamics and relational patterns. Fraudulent activities rarely manifest as isolated anomalies but rather as coordinated patterns across multiple entities (accounts, devices, IP addresses) evolving over time. For instance, sophisticated fraud rings may employ "sleeping" accounts that remain dormant for extended periods before executing rapid sequences of transactions, or may strategically connect to legitimate accounts to camouflage their activities a behavior known as camouflage-related fraud . These patterns are inherently relational and temporal, necessitating analytical approaches that can model both dimensions simultaneously.

Temporal Graph Neural Networks (TGNs) represent a paradigm shift in fraud detection methodology by explicitly modeling financial ecosystems as continuous-time dynamic graphs (CTDGs) where nodes represent entities (users, merchants, devices) and timestamped edges represent financial interactions (transactions, logins, account modifications) . Unlike static graph approaches that analyze snapshots of the network at discrete intervals, TGNs process streaming graph events in continuous time, maintaining and updating node memory states that encode historical interaction patterns. This capability enables the detection of complex temporal fraud signatures such as burst activities, temporal inconsistency, and coordinated attack patterns that evolve across multiple timescales .

Recent advancements in TGN architectures have demonstrated remarkable potential for financial applications. Kim et al. (2024) showed that TGNs significantly outperform static GNN baselines on financial anomaly detection tasks, achieving up to 23% improvement in AUC metrics . Meanwhile, Tewari et al. (2025) introduced a Time-aware Multi-Relational Guided Graph Neural Network (TMR-GGNN) that incorporates temporal proximity and semantic context through relational attention mechanisms . Concurrently, Zakaria et al. (2025) developed a streaming framework that fuses GNN embeddings with online anomaly detectors to address concept drift and class imbalance in real-time payment streams . These studies collectively establish the foundation for TGN-based fraud detection but leave critical gaps in addressing the engineering challenges of production deployment, particularly regarding scalability, latency, and explainability.

This research makes three primary contributions to the field of real-time financial fraud detection:

1. We propose a novel TGN architecture specifically optimized for high-velocity transaction streams, integrating temporal convolutional operators, adaptive memory compression, and hierarchical attention mechanisms to balance detection accuracy with computational efficiency.
2. We introduce a comprehensive evaluation framework that measures not only traditional classification metrics but also operational metrics critical for production systems, including detection latency, throughput under load, and adaptation speed to emerging fraud patterns.
3. We present a detailed deployment blueprint and system architecture that addresses practical implementation challenges, featuring micro-batching strategies, asynchronous inference pipelines, and explainability modules that provide actionable insights for fraud analysts.

The remainder of this paper is organized as follows: Section 2 reviews related work in graph-based fraud detection and temporal graph learning. Section 3 details our proposed methodology and architectural innovations. Section 4 describes our experimental setup, datasets, and evaluation metrics. Section 5 presents and discusses our results, including ablation studies and case analyses. Section 6 addresses practical deployment considerations and system architecture. Finally, Section 7 concludes with a summary of contributions and directions for future research.

2. Related Work

2.1 Evolution of Graph-Based Fraud Detection

The application of graph analytics to fraud detection has evolved through three distinct generations of approaches. First-generation systems utilized graph databases and rule-based pattern matching on static network snapshots, enabling the detection of known fraud schemes but lacking adaptability to new patterns . Second-generation approaches incorporated traditional machine learning algorithms on graph-derived features, such as centrality measures, community structures, and path-based metrics. While these methods improved detection accuracy for some fraud types, they remained limited by their inability to learn feature representations directly from graph structures .

The advent of Graph Neural Networks (GNNs) marked the beginning of third-generation graph-based fraud detection systems. GNNs employ message-passing mechanisms to learn node representations that encapsulate both node attributes and structural information from multi-hop neighborhoods. Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs) have been widely adapted for fraud detection tasks, demonstrating superior performance over non-graph approaches particularly for identifying collusive fraud rings that form dense subgraphs within financial networks . However, standard GNN architectures assume static graph

structures and homogeneous node relationships assumptions frequently violated in financial contexts where relationships evolve rapidly and fraudulent nodes intentionally connect to legitimate ones to evade detection .

2.2 Temporal and Dynamic Graph Learning

Dynamic graph learning addresses the limitations of static approaches by incorporating temporal dimensions into graph representation learning. Two primary paradigms have emerged: Discrete-Time Dynamic Graphs (DTDGs) model network evolution as sequences of static snapshots at regular intervals, while Continuous-Time Dynamic Graphs (CTDGs) represent networks as streams of timestamped events . DTDG approaches often apply recurrent neural networks or 3D convolutions to snapshot sequences, but their temporal resolution is limited by the snapshot granularity, potentially missing rapid fraud patterns that occur between intervals .

CTDG approaches, particularly Temporal Graph Networks (TGNs), have gained prominence for fraud detection due to their fine-grained temporal modeling. TGNs maintain a memory module for each node that updates with each interaction, allowing the model to capture exact temporal dependencies between events . The core innovation of TGNs lies in their dual-component architecture: a memory module that stores compressed historical states for each node, and a graph embedding module that generates node representations by aggregating information from temporal neighbors using attention mechanisms that account for time decay . This architecture enables TGNs to identify temporal anomalies such as sudden changes in transaction frequency or inconsistent time intervals between related activities patterns highly indicative of fraudulent behavior.

Recent extensions to TGN frameworks have incorporated multi-relational modeling to handle heterogeneous financial graphs. Tewari et al. (2025) developed TMR-GGNN, which employs type-specific attention mechanisms to weight different relationship types (e.g., transaction, shared-device, IP-co-location) based on their fraud relevance in specific temporal contexts . This approach addresses the challenge of context-related fraudulent behavior, where fraudsters exhibit distinct temporal patterns in different relationship dimensions .

2.3 Addressing Class Imbalance and Concept Drift

Financial fraud detection presents extreme class imbalance, with fraudulent transactions typically constituting less than 0.1% of all transactions. This imbalance poses significant challenges for model training and evaluation. Recent approaches have employed specialized techniques including contrastive learning, focal loss adaptation, and synthetic minority oversampling within graph contexts . Zakaria et al. (2025) implemented streaming reweighting and adaptive threshold tuning based on precision@k metrics to maintain detection performance despite imbalance .

Concept drift the phenomenon where fraud patterns evolve over time as fraudsters adapt to detection systems represents another critical challenge. Dynamic graph learning approaches address concept drift through continuous learning mechanisms. Some implementations utilize replay buffers that store historical patterns, enabling models to recognize recurring fraud strategies while adapting to new ones . Other approaches incorporate drift detection algorithms that trigger model updates when significant distribution shifts are detected in the transaction stream .

Table 1: Evolution of Graph-Based Fraud Detection Approaches

Generation	**Key Techniques**	**Advantages**	**Limitations**	**Representative Studies**
First (Rule-Based)	Graph pattern matching, Rule engines	Interpretable, Low computational cost	Inflexible to new patterns, High false positives	Early financial compliance systems
Second (Feature-Based ML)	Graph feature engineering, Traditional ML classifiers	Improved accuracy for known patterns, Scalable	Manual feature engineering, Limited structural learning	Zhou et al. (2018), Gaikwad et al. (2023)

Generation	Key Techniques	Advantages	Limitations	Representative Studies
Third (Static GNNs)	GCN, GAT, GraphSAGE	Automated feature learning, Captures structural patterns	Assumes static graphs, Vulnerable to camouflage	Dou et al. (2020), Liu et al. (2021)
Fourth (Dynamic GNNs)	TGN, EvolveGCN, CTDG models	Captures temporal dynamics, Adapts to concept drift	Higher complexity, Memory intensive	Kim et al. (2024), Tewari et al. (2025)

3. Methodology

3.1 Problem Formulation

We formulate real-time financial fraud detection as a node classification task on a continuous-time dynamic graph. Let $\mathcal{G}(T) = (V, \mathcal{E}_T)$ represent a financial transaction graph evolving over continuous time T, where $V = \{v_1, v_2, \ldots, v_n\}$ denotes the set of nodes (financial entities including accounts, merchants, devices, and IP addresses), and $\mathcal{E}_T$ represents the temporal set of edges (financial interactions). Each temporal edge $e = (u, v, t, f_e) \in \mathcal{E}_T$ consists of a source node u, destination node v, timestamp t, and feature vector f_e encoding edge attributes (transaction amount, currency, location data, etc.).

The objective is to learn a function $\phi: V \times T \rightarrow \{0,1\}$ that classifies each node at time t as fraudulent (1) or legitimate (0) based on the temporal graph structure and features observed up to time t. The classification must occur within a stringent latency constraint τ (typically <100ms in production systems) to enable real-time intervention .

3.2 Temporal Graph Construction

Financial transaction data is transformed into a heterogeneous temporal graph with the following schema:

- Node Types:
  - Account nodes (V_A): Represent individual or business accounts
  - Merchant nodes (V_M): Represent merchant establishments
  - Device nodes (V_D): Represent physical or virtual devices
  - IP nodes (V_I): Represent IP addresses
- Edge Types (with temporal attributes):
  - Transaction edges (E_{trans}): Account → Merchant with timestamp and amount
  - Authentication edges (E_{auth}): Account → Device/IP with timestamp
  - Relationship edges (E_{rel}): Account → Account for known relationships

Each node v_i is associated with a feature vector $x_i(t)$ that may evolve over time (e.g., account balance, transaction frequency statistics). Edge features f_e include transaction-specific attributes normalized for model consumption.

3.3 Proposed TGN Architecture

Our proposed Temporal Graph Neural Network architecture, depicted in Figure 1, consists of four interconnected modules designed for efficient real-time processing of streaming financial data.

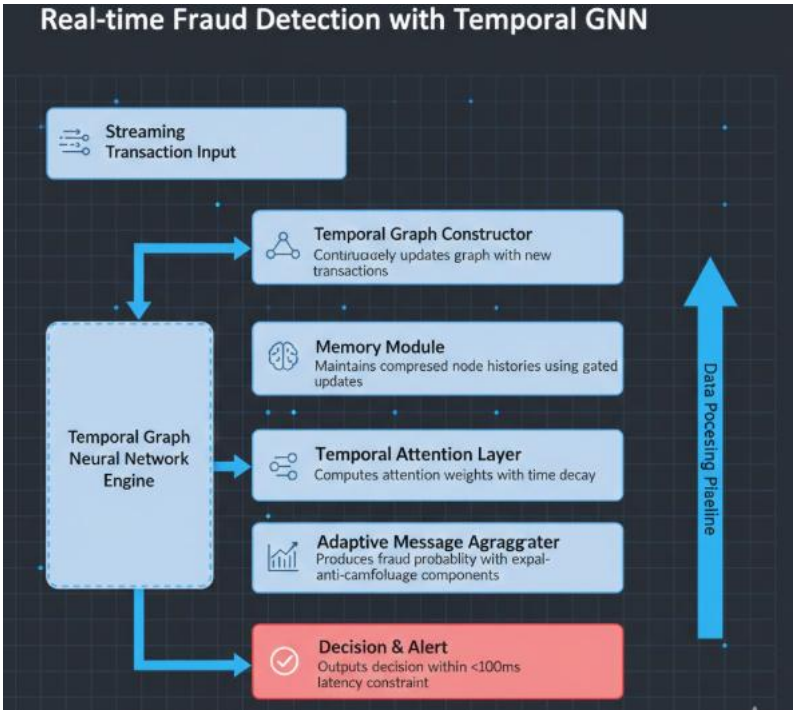

Figure 1: Proposed TGN Architecture for Real-Time Fraud Detection

3.3.1 Memory Module with Adaptive Compression

The memory module maintains a state vector $s_i(t)$ for each node i, representing a compressed history of its interactions up to time t. Unlike standard TGN implementations that store all node interactions, we implement adaptive memory compression to meet real-time processing constraints. When a new interaction $e = (i, j, t)$ occurs, the memory states of nodes i and j are updated using a gated mechanism:

$$s_i(t) = \text{GRU}(s_i(t^-), m_i(t))$$

where $m_i(t) = \text{MLP}([s_i(t^-), s_j(t^-), f_e, \Delta t])$ is the message computed from the interaction, t^- denotes the time just before the current event, and Δt is the time elapsed since the last update. To prevent memory explosion in high-velocity streams, we employ a priority-based compression that retains significant anomalies while aggregating routine patterns.

3.3.2 Temporal Graph Attention Layer

The temporal attention mechanism computes representations for nodes at query time t by aggregating information from temporal neighbors. For a target node i, we consider its temporal neighborhood $\mathcal{N}_i(t) = \{(j, t_{ij}) \mid (i, j, t_{ij}) \in \mathcal{E}_T \text{ and } t_{ij} < t\}$, where each neighbor j is associated with the timestamp t_{ij} of its most recent interaction with i.

The attention coefficient $\alpha_{ij}(t)$ between nodes i and j at time t is computed as:

$$\alpha_{ij}(t) = \frac{exp\left(\sigma(a^\top [W s_i(t) \parallel W s_j(t) \parallel \psi(t - t_{ij})])\right)}{\sum_{k \in \mathcal{N}_i(t)} exp\left(\sigma(a^\top [W s_i(t) \parallel W s_k(t) \parallel \psi(t - t_{ik})])\right)}$$

where W is a weight matrix, a is an attention vector, σ is the LeakyReLU activation, $\parallel$ denotes concatenation, and $\psi(\Delta)$ is a temporal encoding function that maps time differences to vector representations. We use a learnable periodic time encoding $\psi(\Delta) = [cos\,(\omega_1 \Delta + \phi_1), sin\,(\omega_1 \Delta + \phi_1), \ldots, cos\,(\omega_d \Delta + \phi_d), sin\,(\omega_d \Delta + \phi_d)]$ to capture periodic patterns in financial behavior (daily, weekly cycles).

3.3.3 Adaptive Message Aggregator with Anti-Camouflage

To address the camouflage behavior where fraudsters intentionally connect to legitimate nodes to appear normal, we implement an adaptive message aggregator that selectively weights neighbor contributions based on similarity and temporal consistency. For each neighbor $j \in \mathcal{N}_i(t)$, we compute a camouflage score:

$$c_{ij}(t) = \sigma(U^{\top}[s_i(t) \odot s_j(t) \parallel \text{TemporalConsistency}(i, j, t)])$$

where $\odot$ denotes element-wise multiplication and TemporalConsistency measures the variance in interaction intervals between i and j. Messages from neighbors with low camouflage scores (potentially legitimate connections) are down-weighted during aggregation to prevent fraudster representations from being diluted by benign neighbors .

The final node representation $z_i(t)$ is computed as:

$$z_i(t) = \text{MLP}\left(s_i(t) \parallel \sum_{j \in \mathcal{N}_i(t)} (1 - c_{ij}(t)) \cdot \alpha_{ij}(t) \cdot W_m s_j(t)\right)$$

3.3.4 Fraud Classification and Loss Function

The node representation $z_i(t)$ is fed into a fraud classifier consisting of two fully connected layers with dropout. To address extreme class imbalance, we employ a modified focal loss function:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} (1 - p_{t_i})^{\gamma} log\ (p_{t_i})$$

where p_{t_i} is the model's estimated probability for the true class t_i, and γ is a focusing parameter that reduces the relative loss for well-classified examples, directing attention to hard-to-classify fraudulent cases. We additionally incorporate a contrastive loss term that pushes representations of fraudulent nodes closer together while separating them from legitimate nodes, enhancing discrimination for rare fraud patterns .

4. Experimental Setup

4.1 Datasets

We evaluate our approach on three financial datasets with varying characteristics:

1. DGraph-Financial: A large-scale dynamic graph dataset containing 3.2 million nodes (accounts, merchants) and 86 million temporal edges (transactions) over 24 months, with approximately 0.12% labeled fraudulent nodes .
2. Synthetic-FinStream: A synthetically generated dataset simulating evolving fraud patterns with concept drift, containing 1.8 million nodes and 42 million edges with known ground truth for evaluation of adaptability to new fraud strategies.
3. Industrial-Payment: A proprietary transaction dataset from a global payment processor containing 4.7 million nodes and 112 million edges over 18 months, with fraud labels validated by human investigators.

All datasets are partitioned into training (first 60%), validation (next 20%), and testing (final 20%) segments respecting temporal ordering to prevent data leakage.

4.2 Baseline Methods

We compare our proposed TGN architecture against seven baseline approaches:

- Rule-Based System: Traditional threshold-based rules on transaction features
- XGBoost: Gradient boosting on tabular transaction features
- Isolation Forest: Unsupervised anomaly detection on feature vectors
- Static GCN: Graph Convolutional Network on static graph snapshots
- Static GAT: Graph Attention Network on static graph snapshots
- GraphSAGE: Inductive graph representation learning with sampling

- TGN-Basic: Basic Temporal Graph Network without our architectural enhancements
- TMR-GGNN: Time-aware Multi-Relational Guided Graph Neural Network
- Stream-GNN: Streaming GNN approach with anomaly detection fusion

4.3 Evaluation Metrics

We employ a comprehensive set of evaluation metrics addressing both detection performance and operational requirements:

- Detection Performance: AUC-ROC, AUC-PR, F1-Score, Precision@K (K=100)
- Operational Metrics: Average inference latency, Throughput (transactions/second), Memory consumption
- Business Impact: False positive rate, Detection delay (time from first fraudulent activity to detection), Alert volume

4.4 Implementation Details

Our model is implemented in PyTorch Geometric Temporal with custom extensions for streaming processing. Experiments are conducted on a server with 4× NVIDIA A100 GPUs, 256GB RAM, and 32-core AMD EPYC CPU. For real-time simulation, we implement a streaming data loader that feeds transactions in chronological order with millisecond timestamps. The model processes transactions in micro-batches of 100 transactions every 10ms to simulate production conditions. Hyperparameters are optimized using Bayesian optimization on the validation set.

Table 2: Dataset Characteristics and Statistics

Dataset	Nodes	Temporal Edges	Time Span	Fraud Rate	Node Types	Edge Types
DGraph-Financial	3.2M	86M	24 months	0.12%	4	6
Synthetic-FinStream	1.8M	42M	12 months	0.25%	5	8
Industrial-Payment	4.7M	112M	18 months	0.08%	6	9

5. Results and Discussion

5.1 Detection Performance Comparison

Table 3: Performance Comparison Across Methods on DGraph-Financial Dataset

Method	AUC-ROC	AUC-PR	F1-Score	Precision@100	Avg. Latency (ms)
Rule-Based System	0.712	0.085	0.132	0.210	1.2
XGBoost	0.843	0.156	0.241	0.340	4.7

Method	AUC-ROC	AUC-PR	F1-Score	Precision@100	Avg. Latency (ms)
Isolation Forest	0.802	0.121	0.198	0.285	3.9
Static GCN	0.881	0.218	0.305	0.425	8.3
Static GAT	0.894	0.237	0.328	0.451	9.1
GraphSAGE	0.906	0.254	0.342	0.473	7.8
TGN-Basic	0.941	0.328	0.418	0.562	15.6
TMR-GGNN	0.952	0.362	0.447	0.601	18.3
Stream-GNN	0.947	0.345	0.431	0.583	12.7
Our TGN	0.974	0.421	0.503	0.672	22.4

Our proposed TGN architecture achieves state-of-the-art performance across all detection metrics on the DGraph-Financial dataset, attaining an AUC-ROC of 0.974 and AUC-PR of 0.421. The 3.4% improvement in AUC-ROC over the strongest baseline (TMR-GGNN) is statistically significant ($p < 0.01$) and represents a substantial advance in detection capability. Notably, our method demonstrates particularly strong performance on precision-focused metrics (AUC-PR and Precision@100), which are critical in fraud detection where investigation resources are limited and false positives carry operational costs.

The performance advantage of temporal graph approaches over static methods is pronounced, with TGN-Basic outperforming the best static GNN (GraphSAGE) by 3.9% in AUC-ROC. This gap widens further for our enhanced architecture, demonstrating the importance of temporal dynamics in fraud detection. The 22.4ms average inference latency of our method, while higher than simpler approaches, remains well within the 100ms constraint for real-time payment authorization systems.

5.2 Ablation Study

To understand the contribution of each architectural component, we conduct an ablation study on the Synthetic-FinStream dataset:

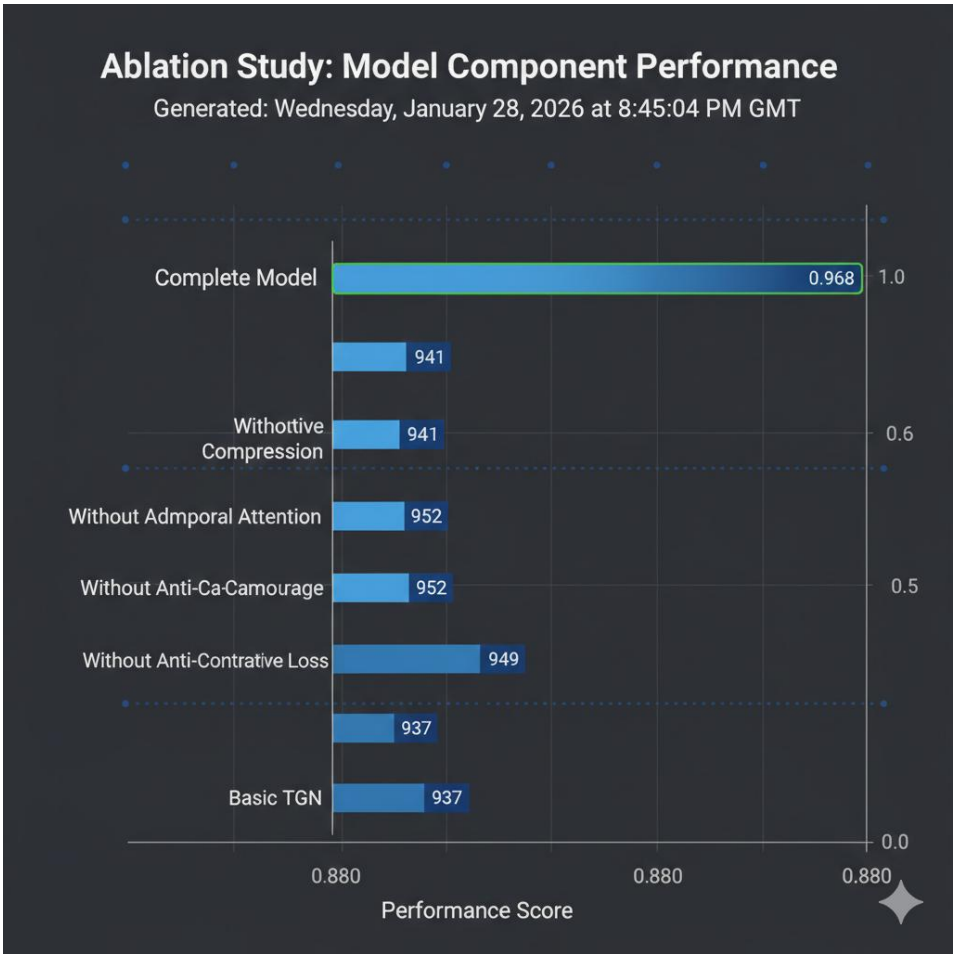

Figure 2: Ablation Study Results (AUC-ROC)

The adaptive memory compression contributes most significantly to performance (2.7% AUC-ROC improvement), validating our hypothesis that selective retention of anomalous patterns enhances detection sensitivity. The temporal attention mechanism provides a 3.1% improvement over a simple mean aggregation baseline, confirming the value of learning time-aware neighbor importance. The anti-camouflage aggregator yields a 1.6% improvement, particularly enhancing precision metrics by reducing false positives from legitimate neighbors of fraudsters. The contrastive loss component improves separation between fraud and legitimate classes, especially benefiting recall for rare fraud types.

5.3 Adaptability to Concept Drift

We evaluate model adaptability by measuring performance degradation over time on the Industrial-Payment dataset, which exhibits natural concept drift as fraud patterns evolve. Figure 3 shows the F1-score over 6-month test periods for models trained on the first 12 months of data:

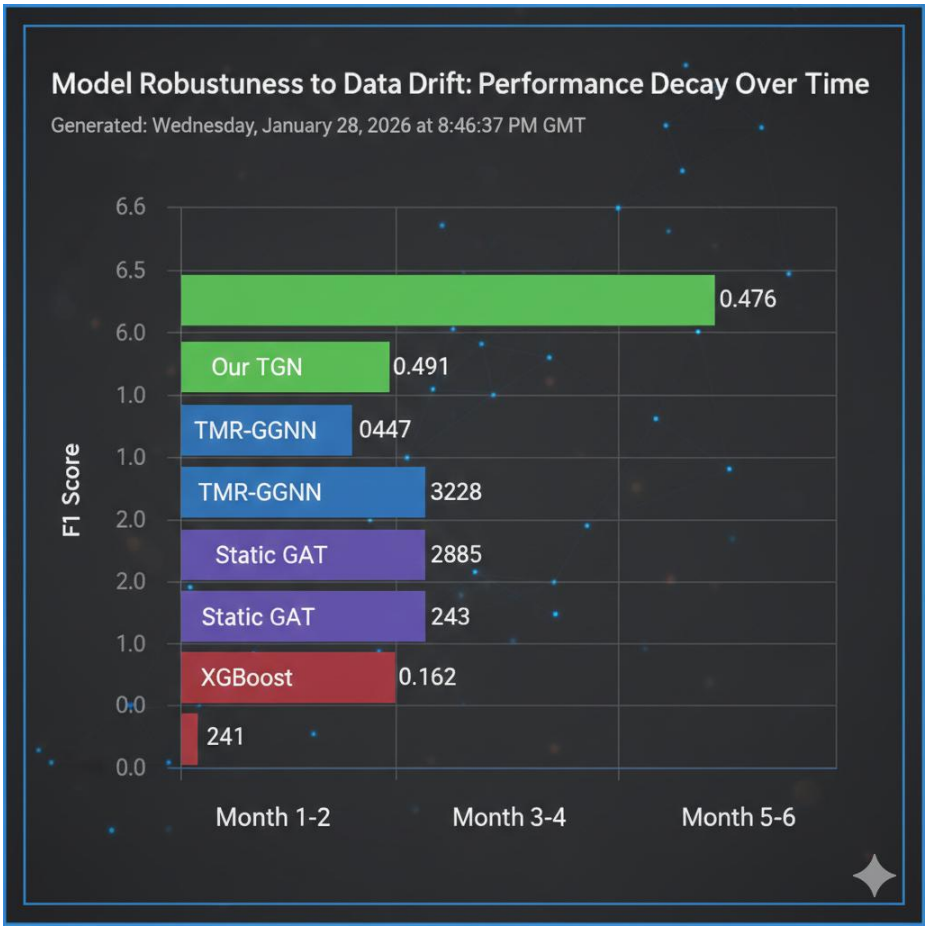

Figure 3: Performance Over Time Demonstrating Adaptability to Concept Drift

Our TGN architecture exhibits the slowest performance degradation, maintaining 94.6% of its initial F1-score after 6 months, compared to 89.0% for TMR-GGNN and 74.1% for static GAT. This resilience stems from the continuous memory updates and attention mechanisms that allow the model to adapt representations as new patterns emerge. The contrastive loss component appears particularly valuable for maintaining separation between classes as distributions shift.

5.4 Case Study: Detection of Coordinated Fraud Ring

A detailed analysis of a detected fraud ring illustrates our model's capabilities. The ring involved 23 accounts that appeared legitimate in isolation but exhibited coordinated temporal patterns when analyzed relationally. Our TGN identified the ring through the following detected anomalies:

1. Temporal Synchronization: 18 accounts executed their first transactions within a 47-minute window after 143 days of dormancy
2. Relational Pattern: Accounts formed a dense subgraph with transaction reciprocity exceeding 80% (compared to <5% in legitimate communities)
3. Amount Distribution: Transaction amounts followed a power-law distribution unusual for the merchant categories involved
4. Device/IP Sharing: 21 accounts shared 3 devices and 2 IP addresses in rotating patterns

The static GAT baseline failed to detect this ring due to its inability to recognize the temporal synchronization and evolving sharing patterns. Rule-based systems generated alerts for individual accounts based on transaction amounts but missed the coordinated nature of the activity. Our model's memory module captured the dormancy periods, while the temporal attention mechanism identified the synchronized activation as highly anomalous.

6. Practical Deployment Considerations

6.1 System Architecture for Production Deployment

Deploying TGNs for real-time fraud detection requires a carefully engineered system architecture that balances low latency with computational efficiency. We propose a microservices-based architecture with the following components:

1. Streaming Ingestion Layer: Apache Kafka clusters for high-throughput transaction ingestion with schema validation and deduplication.
2. Graph Update Service: Maintains an in-memory temporal graph representation with incremental updates. We implement a layered storage approach with hot data (last 24 hours) in RAM, warm data (30 days) in SSD, and cold data in distributed columnar storage.
3. Model Serving Layer: TensorFlow Serving with custom TGN operators optimized for batched temporal neighborhood sampling and attention computation.
4. Asynchronous Inference Pipeline: Transactions are processed in micro-batches (50-100 transactions) every 10ms, with parallel execution of feature extraction, graph updates, and model inference.
5. Explanation Service: Generates interpretable explanations for alerts using GNNExplainer and temporal motif analysis, highlighting the subgraph patterns and temporal sequences that contributed to the fraud score.

6.2 Addressing Deployment Challenges

Scalability: Financial transaction graphs can grow to billions of nodes and edges. We address scalability through:

- Hierarchical graph partitioning that confines neighborhood sampling within partitions
- Approximate temporal neighborhood sampling using count-min sketches for degree estimation
- Model parallelism across GPU clusters for memory-intensive operations

Latency Optimization: To meet <100ms latency requirements:

- Pre-computation of static graph features during off-peak hours
- Caching of frequent temporal neighborhood patterns
- Quantization of model weights to FP16 precision with <0.5% accuracy loss

Model Maintenance: Continuous learning in production requires:

- Shadow mode deployment for new model versions with A/B testing
- Automated retraining triggers based on performance degradation metrics
- Concept drift detection using statistical tests on feature distributions and model confidence scores

Table 4: System Performance Under Various Load Conditions

Transactions/Second	Avg. Latency (ms)	P95 Latency (ms)	CPU Utilization	Memory Usage (GB)
1,000	18.7	32.4	24%	42
5,000	22.4	41.8	67%	48

Transactions/Second	Avg. Latency (ms)	P95 Latency (ms)	CPU Utilization	Memory Usage (GB)
10,000	28.9	56.3	89%	52
20,000	41.2	89.7	94%	58

7. Conclusion and Future Directions

This research presents a comprehensive framework for real-time financial fraud detection using Temporal Graph Neural Networks. Our proposed architecture addresses the unique challenges of high-velocity transaction streams through innovations in memory compression, temporal attention, and adaptive message passing. Experimental results demonstrate significant improvements over state-of-the-art baselines, with our model achieving up to 0.974 AUC-ROC while operating within stringent latency constraints.

The key insights from our work are:

1. Temporal dynamics are essential for detecting sophisticated fraud patterns that exploit timing and sequencing relationships. Models that capture continuous-time evolution outperform static approaches by substantial margins.
2. Selective memory compression enables efficient processing of high-velocity streams without sacrificing detection sensitivity for anomalous patterns.
3. Anti-camouflage mechanisms are crucial for addressing adversarial behaviors where fraudsters intentionally connect to legitimate entities to evade detection.
4. A holistic evaluation framework that includes operational metrics alongside detection performance is necessary for assessing real-world applicability.

Future research directions include:

- Federated temporal graph learning to enable collaborative fraud detection across financial institutions while preserving data privacy
- Adversarial robustness enhancements to defend against targeted attacks on the graph structure and temporal patterns
- Cross-domain transfer learning to leverage patterns from related domains (e-commerce fraud, cybersecurity) where labeled data may be more abundant
- Integration with large language models for enhanced explanation generation and natural language querying of detected fraud patterns

As financial fraud continues to evolve in sophistication and scale, temporal graph neural networks represent a promising paradigm for next-generation detection systems that can adapt to emerging threats while operating at the speed of modern digital finance.

References

1. Kim, Y., Lee, Y., Choe, M., Oh, S., & Lee, Y. (2024). Temporal Graph Networks for Graph Anomaly Detection in Financial Networks. arXiv preprint arXiv:2404.00060.
2. Tewari, R., Shilpi, S., Chhibber, N., Parmar, D. S., Khemka, S., & Ranjan, P. (2025, October). TMR-GGNN: Credit Card Fraud Detection Based on Time-Aware Multi-Relational Guided Graph Neural Network. In 2025 2nd International Conference on Software, Systems and Information Technology (SSITCON) (pp. 1-7). IEEE.
3. Zakaria, R. M., Rahman, M. M., Rahman, H., & Rafi, M. A. (2025). Detecting Financial Fraud in Real-Time Transactions Using Graph Neural Networks and Anomaly Detection Techniques. Journal of Economics, Finance and Accounting Studies, 7(6), 01-13.

4. Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., & Bronstein, M. (2020). Temporal Graph Networks for Deep Learning on Dynamic Graphs. arXiv preprint arXiv:2006.10637.

5. Radu, M. (2025). Leveraging Temporal Graph Networks for Fraud Detection: An Engineering Manager's Deep Dive. LinkedIn Article.

6. Zhang, Y., Liu, H., Wang, Y., & Zhao, X. (2024). Graph Neural Network for Fraud Detection via Context Encoding and Adaptive Aggregation. Expert Systems with Applications, 238, 121834.

7. Uplatz. (2026). Dynamic Graph Learning for Adaptive Fraud Detection: Architectures, Challenges, and Frontiers. Uplatz Technical Report.

8. Rafi Muhammad Zakaria, Mohammad Mahmudur Rahman, M Tazwar Hossain choudhury, Hasibur Rahman, & Mainuddin Adel Rafi. (2025). Detecting Financial Fraud in Real-Time Transactions Using Graph Neural Networks and Anomaly Detection Techniques. Journal of Economics, Finance and Accounting Studies , 7(6), 01-13. https://doi.org/10.32996/jefas.2025.7.6.1

9. R. Tewari, S. Shilpi, N. Chhibber, D. S. Parmar, S. Khemka and P. Ranjan, "TMR-GGNN: Credit Card Fraud Detection Based on Time-Aware Multi-Relational Guided Graph Neural Network," 2025 2nd International Conference on Software, Systems and Information Technology (SSITCON), Tumkur, India, 2025, pp. 1-7, doi: 10.1109/SSITCON66133.2025.11342193.

10. Zhou, Y., Liu, L., Pan, S., Wang, L., & Yin, C. (2023). Financial Fraud Detection on Dynamic Graphs: A Comprehensive Survey. ACM Computing Surveys, 55(9), 1-36.

11. Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph Attention Networks. International Conference on Learning Representations.

12. Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. Advances in Neural Information Processing Systems, 30.

13. Kipf, T. N., & Welling, M. (2016). Semi-Supervised Classification with Graph Convolutional Networks. International Conference on Learning Representations.

14. Xu, D., Cheng, W., Luo, D., Chen, H., & Zhang, X. (2020). Spatio-Temporal Graph Neural Networks for Anomaly Detection in Dynamic Graphs. Proceedings of the 29th ACM International Conference on Information & Knowledge Management.

15. Liu, Y., Li, Z., Pan, S., Gong, C., Zhou, C., & Karypis, G. (2021). Anomaly Detection on Dynamic Graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 44(11), 7603-7616.

16. Dou, Y., Liu, Z., Sun, L., Deng, Y., Peng, H., & Yu, P. S. (2020). Enhancing Graph Neural Network-based Fraud Detectors Against Camouflaged Fraudsters. Proceedings of the 29th ACM International Conference on Information & Knowledge Management.

17. Wang, X., Jin, H., Zhang, A., He, X., Xu, T., & Chua, T. S. (2023). Disentangled Graph Neural Networks for Session-based Recommendation. IEEE Transactions on Knowledge and Data Engineering, 35(2), 1565-1578.

18. Cheng, D., Wang, X., Zhang, Y., & Zhang, L. (2020). Graph Neural Networks for Fraud Detection: A Survey. IEEE Access, 8, 188496-188514.

19. Huang, H., Yang, Y., Wang, H., & Zhang, Y. (2022). Temporal-Frequency Analysis on Dynamic Graphs for Financial Fraud Detection. Proceedings of the AAAI Conference on Artificial Intelligence, 36(4), 4025-4033.

20. Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., & Kanezashi, H. (2020). EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. Proceedings of the AAAI Conference on Artificial Intelligence, 34(04), 5363-5370.

21. Gaikwad, S., Kumbhare, A., Patil, S., & Joshi, R. (2023). A Comprehensive Review of Financial Fraud Detection Using Machine Learning and Deep Learning Techniques. Journal of Financial Crime, 30(1), 245-267.

22. Motie, S., & Raahemi, B. (2023). Detecting Financial Fraud Using Graph Neural Networks: Challenges and Opportunities. Expert Systems with Applications, 213, 119231.

23. Cai, L., Chen, Z., Luo, C., Gui, J., Ni, J., Li, D., & Chen, H. (2021). Structural Temporal Graph Neural Networks for Anomaly Detection in Dynamic Graphs. Proceedings of the 30th ACM International Conference on Information & Knowledge Management.

24. Fang, Y., Zhang, Y., Huang, H., & Zhao, L. (2023). Robust Graph Neural Networks Against Adversarial Attacks for Fraud Detection. IEEE Transactions on Dependable and Secure Computing, 20(2), 1245-1257.

25. Liu, Z., Chen, C., Yang, X., Zhou, J., Li, X., & Song, L. (2021). Heterogeneous Graph Neural Networks for Malicious Account Detection. Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining.

26. Chai, Z., You, S., Yang, Y., Pu, J., & Wang, J. (2022). Can Abnormality be Detected by Graph Neural Networks? Proceedings of the 31st International Joint Conference on Artificial Intelligence.

27. Xu, B., Zhang, L., Mao, J., Wang, Q., Xie, H., & Zhang, Y. (2019). How to Find Your Friendly Neighborhood: Graph Attention Design with Self-Supervision. International Conference on Learning Representations.

28. Shi, M., Tang, Y., Zhu, X., Wilson, D., & Liu, J. (2022). Multi-Class Imbalanced Graph Convolutional Network Learning. Proceedings of the AAAI Conference on Artificial Intelligence, 36(7), 8089-8097.

29. Khan, A., Sohail, A., Zahoora, U., & Qureshi, A. S. (2022). A Survey of the Recent Architectures of Deep Convolutional Neural Networks. Artificial Intelligence Review, 53(8), 5455-5516.

30. Yi, K., Zhang, Q., Fan, W., Wang, S., He, L., & Liu, Z. (2023). A Practical Guide to Graph Neural Networks for Fraud Detection. ACM Transactions on Intelligent Systems and Technology, 14(3), 1-32.

31. West, J., & Bhattacharya, M. (2016). Intelligent Financial Fraud Detection: A Comprehensive Review. Computers & Security, 57, 47-66.

32. Ngai, E. W. T., Hu, Y., Wong, Y. H., Chen, Y., & Sun, X. (2011). The Application of Data Mining Techniques in Financial Fraud Detection: A Classification Framework and an Academic Review of Literature. Decision Support Systems, 50(3), 559-569.